Web Application Security Handbook

Topics: Cookies, Sessions, Authentication Mechanisms

1. Cookies

What Are Cookies?

Cookies are small pieces of data stored on a user's browser by a web server. They are sent back to the server with each subsequent request to help maintain stateful information over stateless HTTP connections.

Why Cookies Are Used

- To identify or recognize users across visits
- To store user preferences such as language or layout settings
- To track behavior for analytics or personalization

Cookie Attributes

Attribute	Description	Example
Name	Identifier of the cookie	`session_id`
Value	Actual data stored in the cookie	`ab123cd`
Expires	Expiry timestamp for the cookie	`Fri, 18 Nov 2025...`
Secure	If `TRUE`, cookie is sent over HTTPS only	`TRUE`
HttpOnly	Prevents JavaScript access to the cookie	`TRUE`
SameSite	Restricts when cookie is sent with requests	`Lax`, `Strict`, etc.

Cookie Workflow Example

1. User logs into example.com
2. Server sends a "Set-Cookie" header: session_id=ab123cd
3. Browser stores the cookie locally
4. On future requests, browser sends this cookie to the server
5. Server uses the cookie to maintain login state or identify the user

2. Sessions

What Is a Session?

A session is a mechanism to store user-specific data on the server. It persists across multiple user requests and helps a web application recognize a user during a visit.

Relationship Between Sessions and Cookies

- When a user logs in, the server creates a session and stores data (e.g., user ID, role).
- The server sends a session ID to the browser using a cookie (e.g., `sessionid=abc123`).
- For each subsequent request, the browser sends the session ID cookie, enabling the server to retrieve the corresponding session data.

Session Workflow Example

1. User logs in
2. Server creates session: {userID: 123, cart: ["Book"]}
3. Server sends cookie: sessionid=abc123
4. Browser stores cookie
5. Browser sends cookie on future requests
6. Server retrieves session data using sessionid and serves content accordingly

---

Comparison: Cookie vs Session

Feature	Cookie	Session
Storage Location	Browser (client-side)	Server (server-side)
Data Capacity	Limited (~4KB)	Higher (depends on server)
Visibility	Visible and editable via browser	Hidden from client
Security Level	Lower (can be stolen via XSS etc.)	Higher (data not exposed)
Example Use Case	“Remember Me” login preference	Shopping cart, user profile

---

3. Authentication Mechanisms

Authentication is the process of verifying the identity of a user in order to control access to an application or service.

1. Basic Authentication

- Credentials (username and password) are sent with each HTTP request, typically in the header.
- No encryption is used unless HTTPS is enabled.
- Considered insecure unless used over HTTPS.

2. Form-Based Authentication

- User submits a login form (username and password).
- Server validates the credentials and creates a session.
- A session ID is stored in a cookie and reused to maintain login state.

3. Token-Based Authentication (e.g., JWT)

- User logs in, and the server returns a digitally signed token (e.g., JSON Web Token).
- The client stores the token (e.g., in local or session storage).
- Token is sent with each request (typically in the Authorization header), allowing stateless authentication.

4. OAuth (Third-Party Authentication)

- User authenticates on a third-party service such as Google or Facebook.
- The external service authenticates the user and returns an access token to the application.
- Used in “Login with Google/Facebook” functionality.

5. Multi-Factor Authentication (MFA)

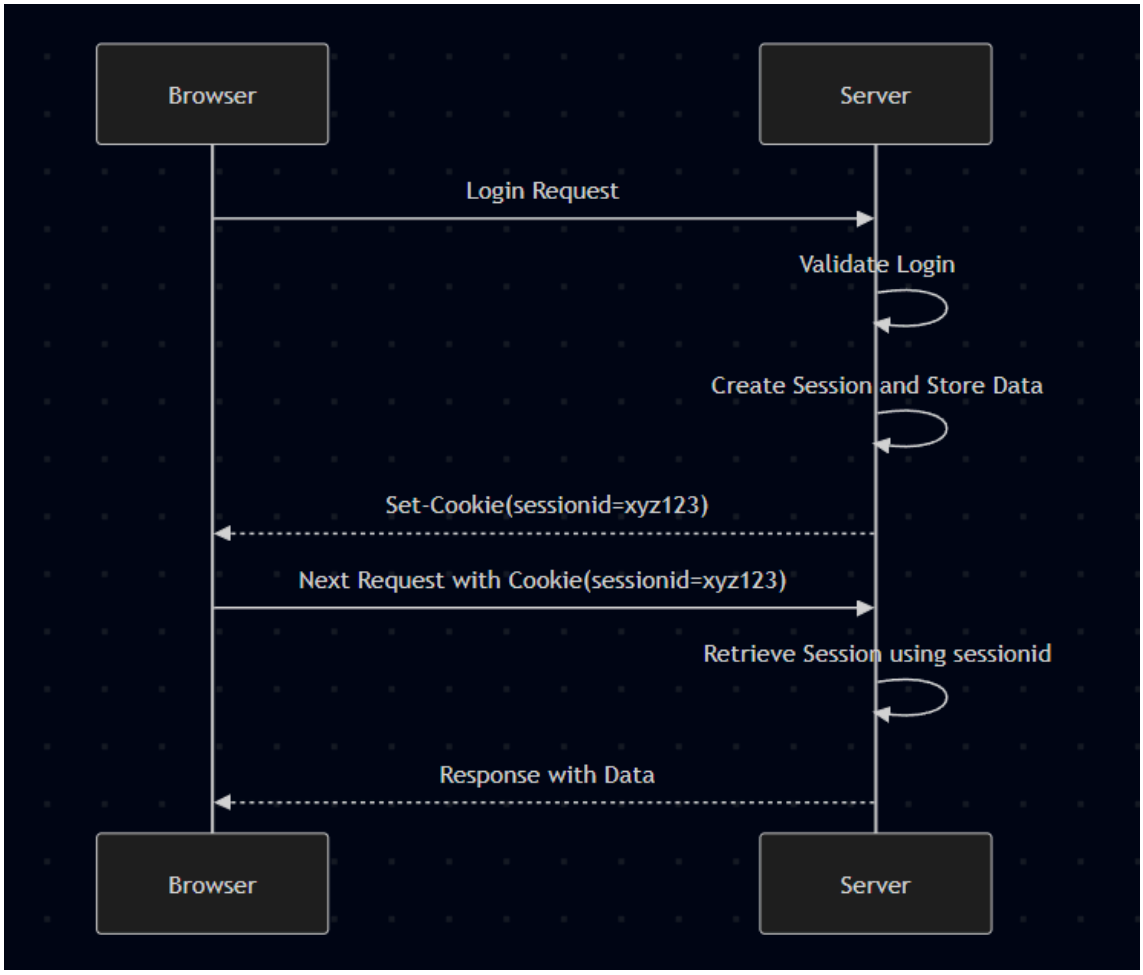
- Requires two or more authentication factors:
  - Something the user knows (e.g., password)
  - Something the user has (e.g., OTP on mobile)
  - Something the user is (e.g., biometric data)
- Ensures stronger security, especially for sensitive applications.

Authentication Summary Table

Method	Authentication Medium	Security Level	Common Use Case
Basic Authentication	HTTP Header	Low	Simple, low-risk APIs
Form-Based Authentication	HTML Form + Cookie/Session	Medium	Web logins
Token-Based (JWT)	Signed Token in Headers	High	APIs, mobile apps, SPAs
OAuth	Third-party Access Token	High	Social login options
Multi-Factor Authentication	Combined methods	Very High	Banking, enterprise applications

---

Diagram: Cookie and Session Interaction



3. Introduction to OWASP Top 10 Security Risks

What is OWASP?

OWASP stands for the **Open Web Application Security Project**, a nonprofit foundation dedicated to improving software security worldwide. Founded in 2001, OWASP provides free, open-source resources, tools, and documentation to help developers, security professionals, and organizations build and maintain secure applications. Think of OWASP as the cybersecurity community's trusted guide for understanding and preventing web application vulnerabilities.

The Purpose of the OWASP Top 10

Every few years, OWASP releases an updated list called the **OWASP Top 10.** a carefully researched ranking of the most critical security risks facing web applications. This list isn't just based on opinion; it's compiled from real-world data contributed by security firms, vulnerability databases, and organizations worldwide. The OWASP Top 10 serves as a security awareness document and a

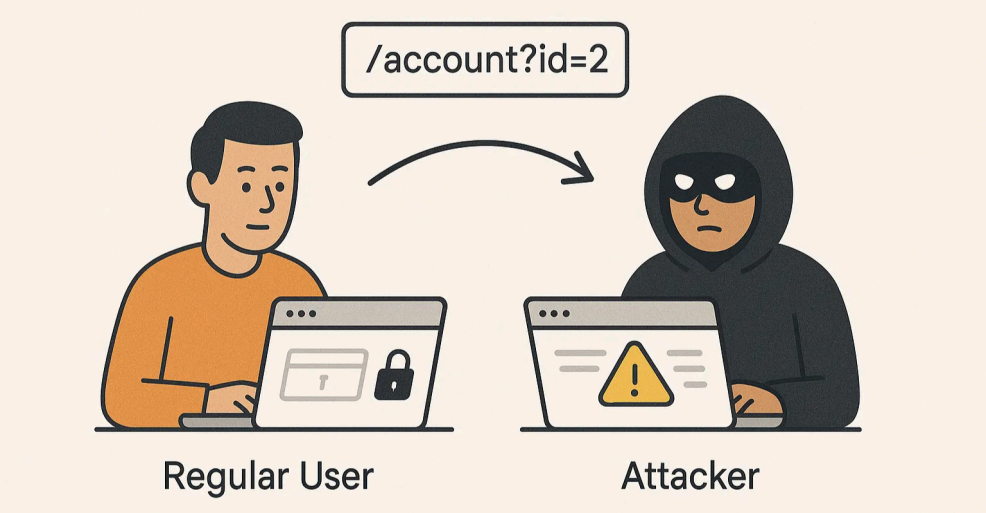
standard for developers, helping them prioritize which vulnerabilities to address first. For students and professionals alike, it's essentially the "must-know" list for web application security.

The OWASP Top 10 Security Risks (2025 Edition)

A01 **Broken Access Control**

Access control enforces policies that prevent users from acting outside their intended permissions. When broken, attackers can access unauthorized functionality or data. like viewing other users' accounts, modifying data they shouldn't touch, or elevating their privileges to admin level. This is the most common and critical vulnerability, occurring when applications fail to properly verify "who can do what."

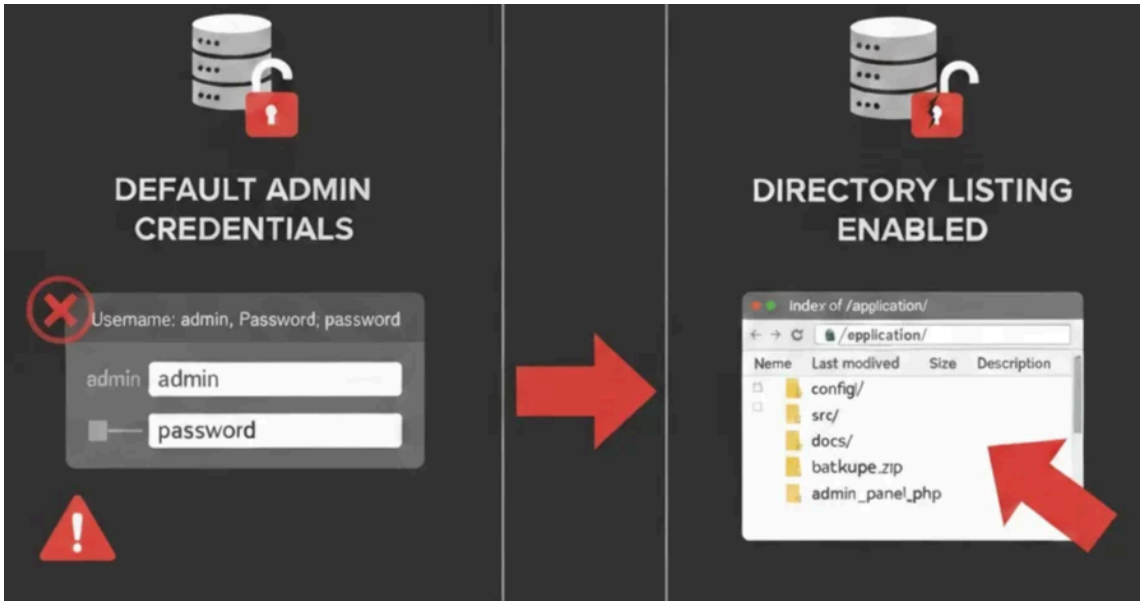
Example: A regular user changing a URL parameter to access another user's private information.



A02 **Security Misconfigurations**

This risk arises from insecure default configurations, incomplete setups, open cloud storage, misconfigured HTTP headers, or verbose error messages containing sensitive information. Modern applications involve multiple layers (networks, platforms, web servers, databases, frameworks), and each must be configured securely. Missing security patches or unnecessary features left enabled are common culprits.

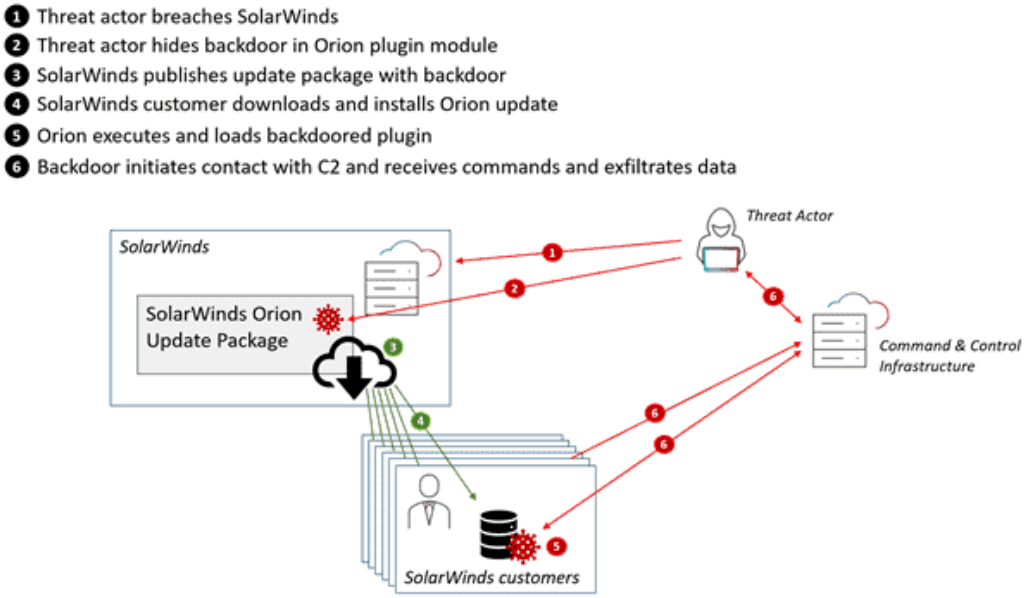
Example: Leaving default admin credentials unchanged or enabling directory listing that exposes the application's file structure.



A03 Software Supply Chain Failures

Software supply chain failures happen when applications rely on components, libraries, services, or models that are compromised, outdated, or improperly verified. These weaknesses are not inherent in your code, but rather in the software and tools you depend on. Attackers exploit these weak links to inject malicious code, bypass security, or steal sensitive data.

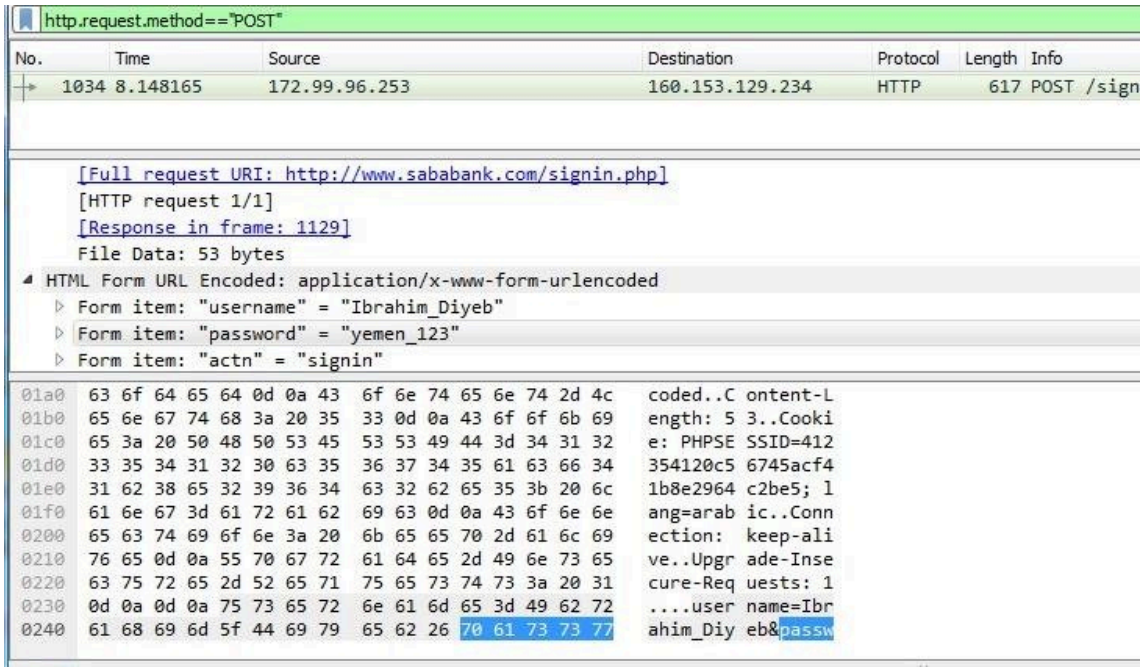
Example: In 2021, the SolarWinds Orion compromise showed the danger of supply chain attacks. Attackers inserted malicious code into a trusted update, affecting thousands of organisations that automatically installed it. This wasn't a bug in SolarWinds' core logic. It was a flaw in the software update building, verification, and distribution process.



A04 **Cryptographic Failures**

Previously known as "Sensitive Data Exposure," this risk involves failures related to cryptography that lead to exposure of sensitive data. This includes transmitting data in cleartext (unencrypted), using weak encryption algorithms, or improperly storing passwords and credit card information. Even if you collect data securely, poor cryptographic practices can expose it to attackers.

Example: A website transmitting passwords over HTTP instead of HTTPS, allowing attackers to intercept credentials.



A05 **Injection**

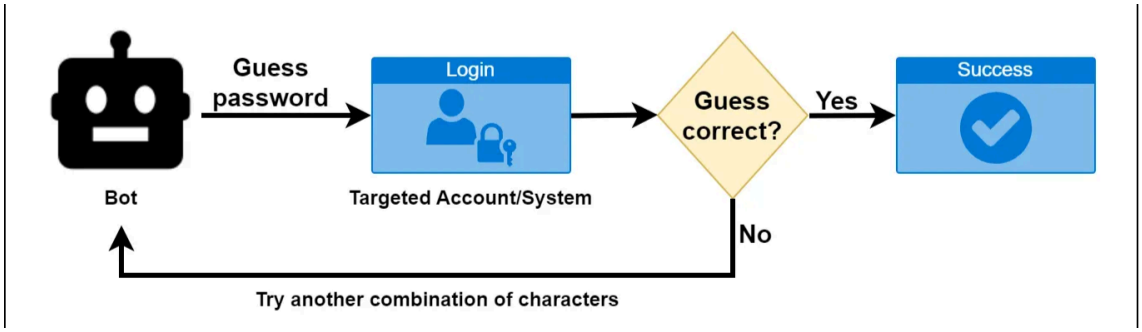
Injection flaws occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data tricks the interpreter into executing unintended commands or accessing unauthorized data. SQL injection, NoSQL injection, and command injection are common examples. This happens when applications don't properly validate, filter, or sanitize user input.

Example: An attacker entering `' OR '1'='1' --` into a login form to bypass authentication by manipulating the SQL query.

A06 **Insecure Design**

This is a broad category focusing on risks related to design and architectural flaws—weaknesses that occur before a single line of code is written. Unlike implementation bugs, insecure design means the application's blueprint itself is flawed. It emphasizes the need for threat modeling, secure design patterns, and reference architectures during the planning phase.

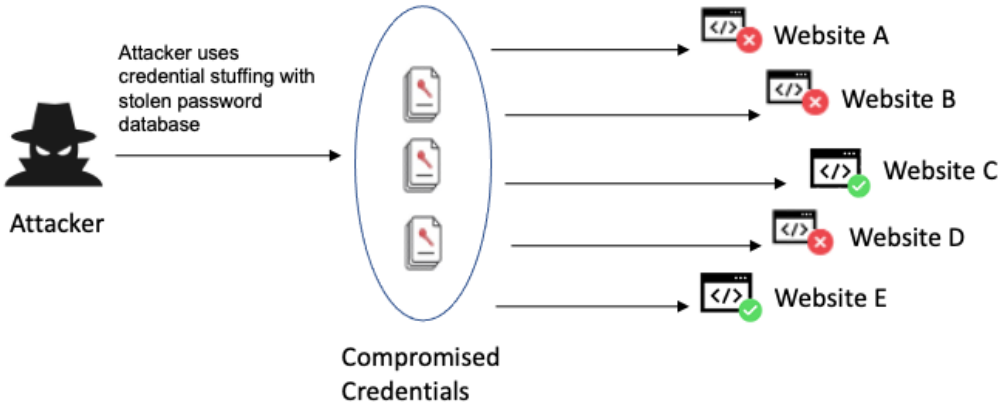
Example: An application that allows unlimited password reset attempts without rate limiting, enabling brute-force attacks by design.



A07 **Authentication Failures**

Previously called "Broken Authentication," this category covers weaknesses in confirming user identity, authentication, and session management. These failures allow attackers to compromise passwords, keys, session tokens, or exploit other implementation flaws to assume other users' identities temporarily or permanently.

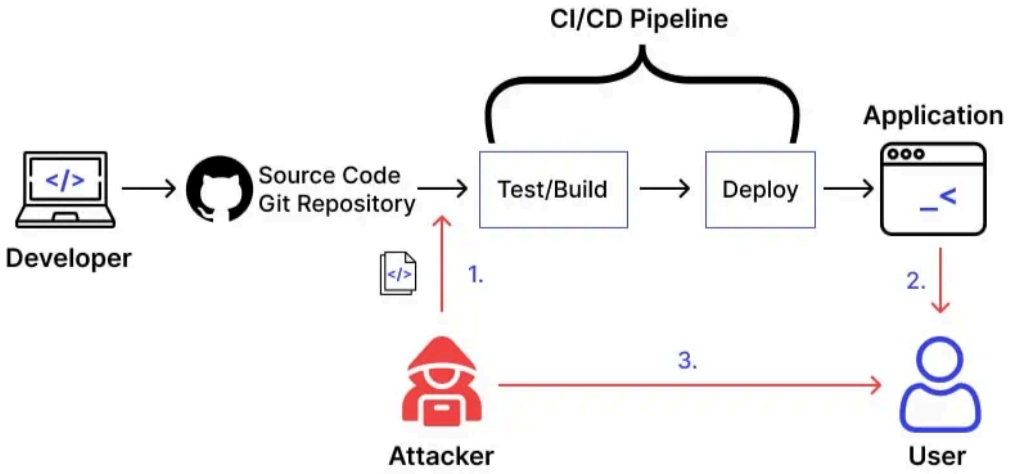
Example: An application allowing weak passwords like "123456" or not implementing multi-factor authentication for sensitive accounts.



A08 **Software or Data Integrity Failures**

This new category addresses code and infrastructure that doesn't protect against integrity violations. This includes insecure CI/CD pipelines, auto-update functionality without integrity verification, and deserializing untrusted data. Attackers can push malicious updates or inject hostile objects into the application.

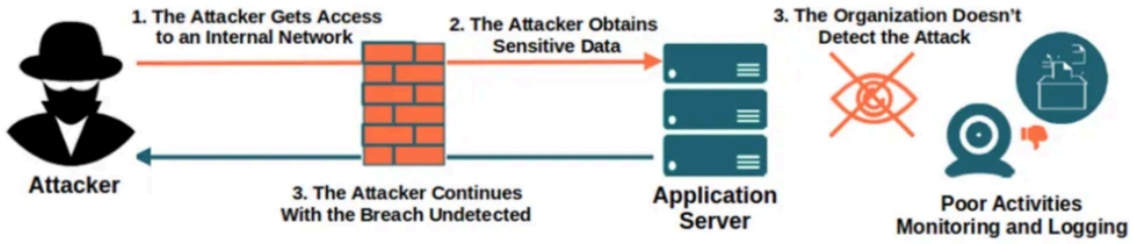
Example: An application downloading and executing plugins from a CDN without verifying the cryptographic signature, allowing attackers to inject malware.



A09 **Logging and Alerting Failures**

Without proper logging and monitoring, breaches cannot be detected in a timely manner. This risk occurs when applications fail to log security events, produce unclear log messages, don't monitor logs for suspicious activity, or can't trigger alerts for active attacks. Even after a breach occurs, inadequate logging makes incident response and forensics nearly impossible.

Example: An application that doesn't log failed login attempts, preventing detection of brute-force attacks in progress.



A10 **Mishandling of Exceptional Conditions**

This category refers to failures in properly anticipating, detecting, or managing unexpected states or error conditions within a system. When applications do not handle exceptional conditions appropriately such as invalid inputs, timeouts, resource limitations, or unexpected system responses they may behave unpredictably or insecurely.

Improper handling of these conditions can result in:

- Unauthorized disclosure of system or application information
- Unexpected termination or disruption of service
- Data corruption or loss
- Opportunities for malicious exploitation

Effective systems are designed to manage exceptional events safely and consistently. This includes validating inputs, implementing secure error-handling procedures, avoiding the exposure of sensitive system details in error messages, and ensuring that the application returns to a stable and predictable state after an exception occurs.